

UNIVERSIDAD TECNOLÓGICA DE TEHUACÁN

TECNOLOGÍAS DE LA INFORMACIÓN DESARROLLO DE SOFTWARE

EJEMPLO INTEGRADOR S8

Animaciones accesibles y optimización de
performance

PRESENTA:

OSCAR ESPINOZA LANDETA

8 “A”

A CARGO:

ING. JOSÉ MIGUEL CARRERA PACHECO



Reporte de Análisis: Animaciones en Interfaces Web

Fecha: 1 de marzo de 2026

Tema: Impacto, rendimiento y mejores prácticas en la animación de interfaces web (UI).

1. Diferencia entre animaciones CSS y JavaScript

La principal diferencia radica en cómo se declaran y cómo el navegador las procesa:

- **Animaciones CSS:** Son declarativas. Se definen en la hoja de estilos mediante transiciones (transition) o fotogramas clave (@keyframes).
 - **Ventajas:** El navegador puede optimizarlas fácilmente. A menudo se ejecutan en un hilo separado (el hilo del compositor), lo que significa que pueden seguir funcionando fluidamente incluso si el hilo principal de JavaScript está bloqueado haciendo tareas pesadas.
 - **Casos de uso ideales:** Estados de interacción simples (hover, focus), revelación de menús, *spinners* de carga y transiciones de estado directas.
- **Animaciones JavaScript:** Son imperativas. Se controlan mediante código (ya sea manipulando el DOM, usando requestAnimationFrame, o mediante la Web Animations API/librerías como GSAP).
 - **Ventajas:** Ofrecen un control absoluto. Permiten pausar, revertir, buscar un punto exacto en el tiempo, sincronizar múltiples animaciones complejas, o basar el movimiento en físicas (como resortes o inercia).
 - **Casos de uso ideales:** Animaciones complejas de múltiples pasos, juegos web, animaciones basadas en el desplazamiento del usuario (*scroll*), o elementos que reaccionan de forma dinámica a la entrada del mouse/táctil.

2. Impacto en la Experiencia de Usuario (UX)

Las animaciones son una herramienta de diseño poderosa que puede ser un arma de doble filo.

¿Cuándo mejoran la experiencia?

- **Proporcionan contexto espacial:** Ayudan al usuario a entender de dónde vienen y a dónde van los elementos (ej. un modal que se expande desde un botón).
- **Dan *feedback* inmediato:** Informan que una acción fue recibida (ej. un botón que se hunde ligeramente al presionarlo o un mensaje de error que tiembla).
- **Guían la atención:** Atraen el ojo hacia un cambio importante en la pantalla sin ser intrusivos.
- **Mejoran el rendimiento percibido:** Los *skeleton screens* (esqueletos de carga animados) hacen que la espera parezca más corta que una pantalla en blanco.

¿Cuándo la perjudican?

- **Cuando son demasiado lentas:** Si el usuario tiene que esperar a que termine una animación para poder interactuar, la aplicación se sentirá lenta y frustrante.
- **Sobrecarga cognitiva:** Animar demasiados elementos al mismo tiempo distrae y confunde.
- **Falta de propósito:** "Animar por animar" (solo para que se vea bonito) suele añadir fricción innecesaria al flujo de trabajo del usuario.

3. ¿Qué es prefers-reduced-motion y por qué es importante?

prefers-reduced-motion es una característica de medios (Media Query) de CSS que detecta si el usuario ha configurado su sistema operativo para minimizar la cantidad de movimiento o animación no esencial.

- **Importancia:** Es una cuestión crítica de **accesibilidad**. Las animaciones extensas, los efectos de paralaje (*parallax*) o los movimientos rápidos pueden causar mareos, náuseas o desencadenar vértigo en personas con trastornos vestibulares.
- **Implementación:** Los desarrolladores deben usar esta media query para desactivar o simplificar drásticamente las animaciones cuando el usuario lo solicite.

/* Ejemplo de implementación */

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation-duration: 0.01ms !important;  
    animation-iteration-count: 1 !important;  
    transition-duration: 0.01ms !important;  
    scroll-behavior: auto !important;  
  }  
}
```

4. Impacto en el Rendimiento de la Aplicación Web

Las animaciones pueden hacer que una aplicación se sienta fluida (60 fotogramas por segundo) o torpe ("janky"). Todo depende de **qué propiedades** se estén animando.

El navegador procesa los cambios visuales en tres pasos: **Layout** (Geometría/Diseño) -> **Paint** (Pintura/Píxeles) -> **Composite** (Composición/Capas).

- **Alto impacto (Malo para el rendimiento):** Animar propiedades que desencadenan *Layout* o *Paint*, como width, height, margin, top, left, o box-shadow. Esto obliga a la CPU a recalcular la posición de todos los elementos en la página en cada fotograma, lo que gasta batería y causa saltos visuales.

- **Bajo impacto (Bueno para el rendimiento):** Animar propiedades que solo requieren *Composite*, principalmente **transform** (traslación, escala, rotación) y **opacity**. Estas propiedades se delegan a la GPU (Aceleración de Hardware), logrando animaciones extremadamente suaves sin bloquear el hilo principal.

5. Errores Comunes al Animar Interfaces

1. **Animar las propiedades equivocadas:** Usar `top` o `left` en lugar de `transform: translate()` es el error de rendimiento más común.
2. **Tiempos incorrectos (Demasiado lento):** Las animaciones de UI deben ser rápidas y ágiles. Por lo general, entre 150ms y 300ms es lo ideal para elementos pequeños. Más de 500ms suele sentirse como una eternidad.
3. **Usar un ritmo lineal (ease-linear):** Los objetos en el mundo real no se mueven a una velocidad constante; aceleran y desaceleran. Usar `linear` hace que la animación se sienta robótica. Es mejor usar curvas de aceleración naturales (`ease-in-out`, `ease-out`).
4. **Bloquear la interacción:** No permitir que el usuario haga clic en un botón o cierre un menú hasta que su animación de entrada haya terminado completamente.
5. **Ignorar la accesibilidad:** No incluir soporte para `prefers-reduced-motion`, excluyendo a usuarios con sensibilidades al movimiento.